

## 实验题目：图像处理

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 9 月 6 日

## 一、实验目的

1. 了解计算机处理和存储图像的基础知识。
2. 掌握 JPEG 标准的基本原理、变换域编码和量化的基本思想，以及 MATLAB 处理矩阵和图像的常用命令。
3. 了解一种在变换域进行信息隐藏的方法。

## 二、基础知识

### 1. 了解图像处理工具箱中函数的基本功能

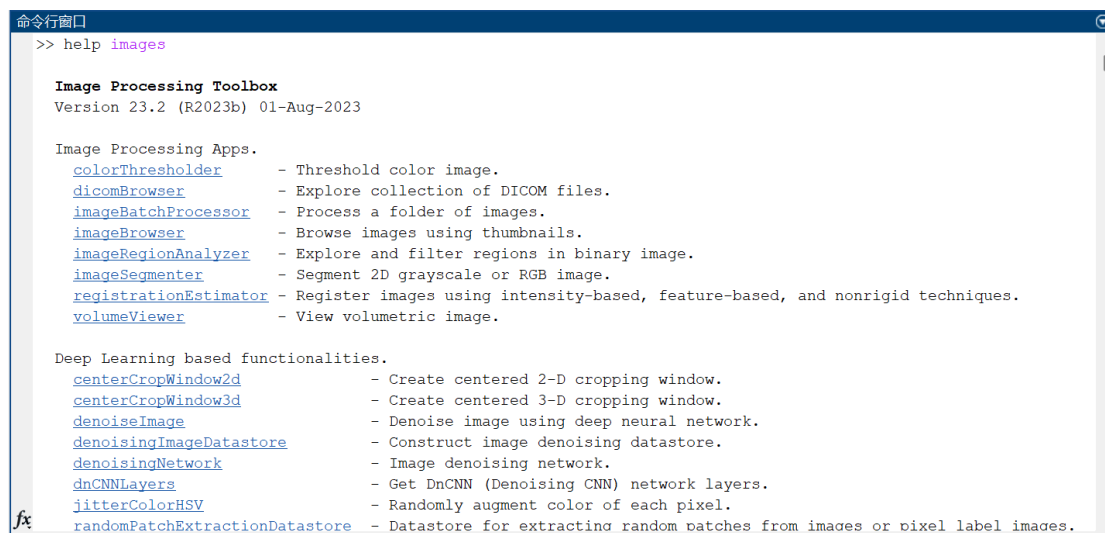
#### (1) 设计思路

MATLAB 工具箱内的函数可通过 help 功能查看。

#### (2) 核心代码

在命令窗口输入 help images。

#### (3) 运行结果



```
命令窗口
>> help images

Image Processing Toolbox
Version 23.2 (R2023b) 01-Aug-2023

Image Processing Apps.
colorThresholder - Threshold color image.
dicomBrowser - Explore collection of DICOM files.
imageBatchProcessor - Process a folder of images.
imageBrowser - Browse images using thumbnails.
imageRegionAnalyzer - Explore and filter regions in binary image.
imageSegmenter - Segment 2D grayscale or RGB image.
registrationEstimator - Register images using intensity-based, feature-based, and nonrigid techniques.
volumeViewer - View volumetric image.

Deep Learning based functionalities.
centerCropWindow2d - Create centered 2-D cropping window.
centerCropWindow3d - Create centered 3-D cropping window.
denoiseImage - Denoise image using deep neural network.
denoisingImageDatastore - Construct image denoising datastore.
denoisingNetwork - Image denoising network.
dnCNNLayers - Get DnCNN (Denoising CNN) network layers.
jitterColorHSV - Randomly augment color of each pixel.
randomPatchExtractionDatastore - Datastore for extracting random patches from images or pixel label images.
```

### 2. 在测试图像上绘制红色圆/黑白格

#### (1) 设计思路

通过分离通道，分别设置红色圆圈（实际是宽度很小的圆环）和黑色方格，并指定其出现的位置，并用 cat 函数将三个通道的信息拼接起来，最后调用 imshow 函数显示图像。

## (2) 核心代码

```
load ('hall.mat');
[h, w, c] = size(hall_color);
[x, y] = meshgrid(1:w, 1:h);

%% 红圈
r = min(h, w) / 2; % 半径
circen = [w, h] / 2; % 中心位置
r_circle = (0.96 * r <= sqrt((x - circen(1)).^2 + (y - circen(2)).^2))
& (sqrt((x - circen(1)).^2 + (y - circen(2)).^2) <= r); % 设置圆环宽度为
图像半径的 4%

% 分离 RGB 通道
r = hall_color(:,:,1);
g = hall_color(:,:,2);
b = hall_color(:,:,3);
% 把圆环区域设置为红色
r(r_circle) = uint8(255);
g(r_circle) = uint8(0);
b(r_circle) = uint8(0);
im1 = cat(3, r, g, b);

figure;
imshow(im1, 'InitialMagnification', 'fit')

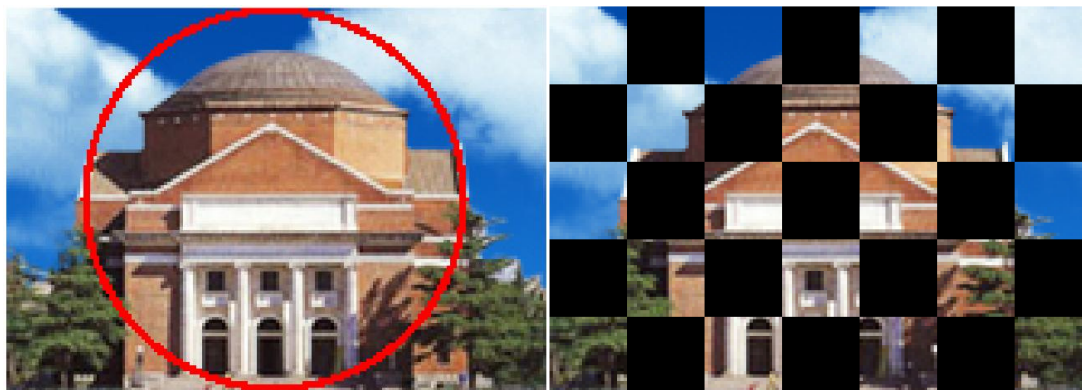
%% 黑白格
pw = round(w/7); % 水平方向分 7 块
ph = round(h/5); % 竖直方向分 5 块
block = xor(mod(ceil(x/pw), 2), mod(ceil(y/ph), 2)); % 划分棋盘格

% 分离 RGB 通道
r = hall_color(:,:,1);
g = hall_color(:,:,2);
b = hall_color(:,:,3);
% 把棋盘格区域内设置黑色
r(block) = uint8(0);
g(block) = uint8(0);
b(block) = uint8(0);
im2 = cat(3, r, g, b);

figure;
imshow(im2, 'InitialMagnification', 'fit')
```

### (3) 运行结果

生成了符合预期的图像。



## 三、 图像压缩编码

### 1. 两种图像预处理方法比较

#### (1) 设计思路

在空域和变换域上分别进行图像预处理操作：在空域上直接减去 128，在变换域上需减去  $128 \times \text{子块的边长}$ （二维 DCT 变换的系数  $1/4 \times 64$  个像素  $\times 128$ ）。计算两种预处理方法的最大绝对误差，若小于阈值即认为等价。

#### (2) 核心代码

```
load('hall.mat');
hall_test = double(hall_gray(80:87, 56:63)); % 8x8 子块

% 空域
im1 = hall_test - 128; % 直接减去 128
dctim1 = dct2(im1);

% 变换域
dctim2 = dct2(hall_test);
dctim2(1,1) = dctim2(1,1) - 128 * 8; % 需要减去 128 * 8

err = max(abs(dctim1(:) - dctim2(:))); % 计算最大绝对误差
fprintf('最大绝对误差 = %.5e\n', err);

if err < 1e-12
    disp('等价');
```

```

else
    disp('不等价');
end

%% 显示部分
figure;
imshow(uint8(hall_test), 'InitialMagnification', 'fit');
title('8 * 8 子块');

figure;
imshow(log(abs(dctim1)+1), [], 'InitialMagnification', 'fit');
title('空域 DCT');

figure;
imshow(log(abs(dctim2)+1), [], 'InitialMagnification', 'fit');
title('变换域 DCT');

```

### (3) 运行结果

```

>> HW2_2_1
最大绝对误差 = 1.42109e-13
等价

```

两种预处理方法等价，说明预处理步骤能在变换域上进行。

## 2. 两种二维 DCT 函数比较

### (1) 设计思路

依据指导书中对于二维 DCT 函数的定义，手动编程实现 DCT 函数。分别调用自带的库函数 `dct2` 和编写的函数 `my_dct2`，计算两种 DCT 函数处理后系数矩阵的最大绝对误差，若小于阈值即认为等价。

$$\mathbf{c} = \mathbf{D}\mathbf{p} = \sqrt{\frac{2}{N}} \begin{bmatrix} \sqrt{\frac{1}{2}} & \sqrt{\frac{1}{2}} & \cdots & \sqrt{\frac{1}{2}} \\ \cos \frac{\pi}{2N} & \cos \frac{3\pi}{2N} & \cdots & \cos \frac{(2N-1)\pi}{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \cos \frac{(N-1)\pi}{2N} & \cos \frac{(N-1)3\pi}{2N} & \cdots & \cos \frac{(N-1)(2N-1)\pi}{2N} \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{N-1} \end{bmatrix}$$

$$\mathbf{C} = \mathbf{D}\mathbf{P}\mathbf{D}^T$$

## (2) 核心代码

```
load ('hall.mat');

hall_test = double(hall_gray(80:87, 56:63)); % 8×8 子块
im = hall_test - 128;

dctim1 = dct2(im);
dctim2 = my_dct2(im);

err = max(abs(dctim1(:) - dctim2(:))); % 计算最大绝对误差
fprintf('最大绝对误差 = %.5e\n', err);

if err < 1e-12
    disp('等价');
else
    disp('不等价');
end

%% 显示部分
figure;
imshow(log(abs(dctim1)+1), [], 'InitialMagnification', 'fit');
title('自带库函数 DCT');

figure;
imshow(log(abs(dctim2)+1), [], 'InitialMagnification', 'fit');
title('编程实现 DCT');

%% 二维 DCT
function C = my_dct2(P)
    [h, w] = size(P);
    assert(w == h, "不是方阵，输入非法！");
    d1 = 1 : 2 : 2 * w - 1;
    d2 = (1 : w - 1)';
    D = cos(d1 .* d2 * pi / (2 * w));
    D = [ones(1, w) * sqrt(1/w); D * sqrt(2/w)];
    C = D * P * D';
end
```

## (3) 运行结果

```
>> HW2_2_2
```

```
最大绝对误差 = 9.94760e-14
```

```
等价
```

两种 DCT 函数等价。

### 3. 将 DCT 系数矩阵的左/右侧四列置零

#### (1) 设计思路

对 DCT 系数矩阵函数进行修改，分别定义左侧四列置零和右侧四列置零的 DCT 系数矩阵。选取一个 8\*8 子块单独处理后，发现不能明显看出结论。因此改为采用分块处理整个图像，经过逆变换并拼合的方法，得到全图的不同种还原结果。

#### (2) 核心代码

```
load('hall.mat');
im = double(hall_gray) - 128;

%% 对全图操作
dct_left = @(P) set_left(dct2(P));
dct_right = @(P) set_right(dct2(P));

% 分块处理整个图像
C1 = blockproc(im, [8, 8], @(block) dct2(block.data));
C2 = blockproc(im, [8, 8], @(block) dct_left(block.data));
C3 = blockproc(im, [8, 8], @(block) dct_right(block.data));

% 逆变换
im1 = uint8(blockproc(C1, [8, 8], @(block) idct2(block.data)) + 128);
im2 = uint8(blockproc(C2, [8, 8], @(block) idct2(block.data)) + 128);
im3 = uint8(blockproc(C3, [8, 8], @(block) idct2(block.data)) + 128);

% 显示结果
figure;
subplot(2,2,1); imshow(hall_gray); title('原始图片');
subplot(2,2,2); imshow(im1); title('原始 DCT 还原');
subplot(2,2,3); imshow(im2); title('左侧四列置 0 的 DCT 还原');
subplot(2,2,4); imshow(im3); title('右侧四列置 0 的 DCT 还原');

%% 选取一块操作
block = im(80:87, 56:63);

% 对选定的块进行 DCT 变换和处理
dct_ori = dct2(block);
dct_left = set_left(dct_ori);
dct_right = set_right(dct_ori);

% 逆变换
block = uint8(block + 128);
```

```
im4 = uint8(idct2(dct_ori) + 128);
im5 = uint8(idct2(dct_left) + 128);
im6 = uint8(idct2(dct_right) + 128);

% 显示结果
figure;
subplot(2,2,1); imshow(block); title('原始图像块');
subplot(2,2,2); imshow(im4); title('原始 DCT 还原');
subplot(2,2,3); imshow(im5); title('左侧四列置 0 的 DCT 还原');
subplot(2,2,4); imshow(im6); title('右侧四列置 0 的 DCT 还原');

%% 函数声明
function C = set_left(C)
    C(:, 1:4) = 0;
end

function C = set_right(C)
    C(:, 5:8) = 0;
end
```

### (3) 运行结果

原始图片



原始DCT还原



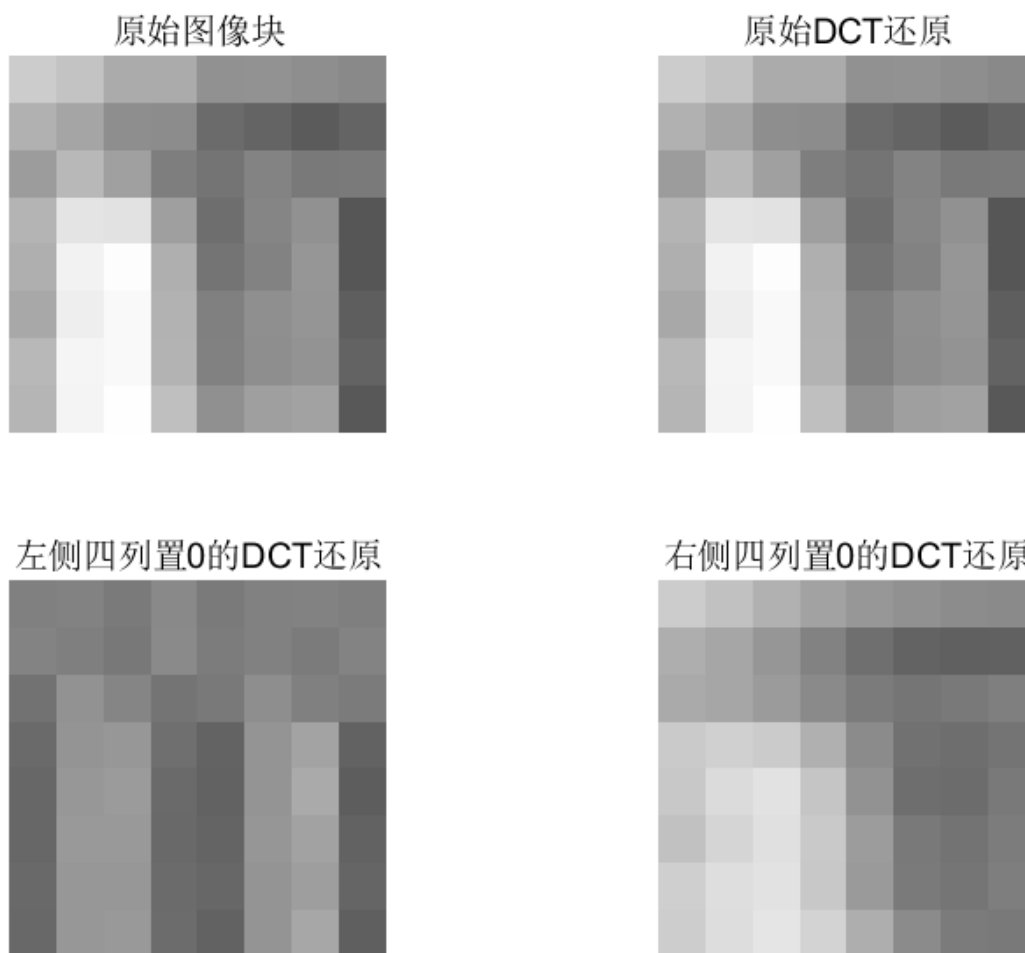
左侧四列置0的DCT还原



右侧四列置0的DCT还原







DCT 系数矩阵左上方的值对应低频分量，右下方的值对应高频分量。而对于一般图像，能量集中在低频直流分量上，DCT 系数矩阵左上方的值较大，右下方的值较小。因此，将左侧四列（低频分量）置零导致图像主要信息丢失，只保留了部分边缘内容；而将右侧四列（高频分量）置零仍能保留主要轮廓，但部分图像细节丢失。主要轮廓是否保留在完整图像的还原图中能明显看出，细节是否丢失在  $8 \times 8$  子块的还原图中更容易看出。

#### 4. 对 DCT 系数做转置、旋转 $90^\circ$ 、旋转 $180^\circ$ 操作

##### (1) 设计思路

保留上一小问的主函数架构基本不变，主要采用分块操作并利用逆变换拼回完整图像的方法，辅以选择子块操作观察细节。声明不同的 DCT 函数，通过 MATLAB 中矩阵的基本运算功能实现系数矩阵的转置（`'`）和旋转（`rot90`）。最终使用不同的 DCT 系数进行变换，并分别输出对应图像。

## (2) 核心代码

```
load('hall.mat');
im = double(hall_gray) - 128;

%% 对全图操作
dct_t = @(P) set_t(dct2(P));
dct_90 = @(P) set_90(dct2(P));
dct_180 = @(P) set_180(dct2(P));

% 分块处理整个图像
C1 = blockproc(im, [8, 8], @(block) dct_t(block.data));
C2 = blockproc(im, [8, 8], @(block) dct_90(block.data));
C3 = blockproc(im, [8, 8], @(block) dct_180(block.data));

% 逆变换
im1 = uint8(blockproc(C1, [8, 8], @(block) idct2(block.data)) + 128);
im2 = uint8(blockproc(C2, [8, 8], @(block) idct2(block.data)) + 128);
im3 = uint8(blockproc(C3, [8, 8], @(block) idct2(block.data)) + 128);

% 显示结果
figure;
subplot(2,2,1); imshow(hall_gray); title('原始图片');
subplot(2,2,2); imshow(im1); title('转置 DCT 还原');
subplot(2,2,3); imshow(im2); title('旋转 90°的 DCT 还原');
subplot(2,2,4); imshow(im3); title('旋转 180°的 DCT 还原');

%% 选取一块操作
block = im(80:87, 56:63);

% 对选定的块进行 DCT 变换和处理
dct_t = set_t(dct2(block));
dct_90 = set_90(dct2(block));
dct_180 = set_180(dct2(block));

% 逆变换
block = uint8(block + 128);
im4 = uint8(idct2(dct_t) + 128);
im5 = uint8(idct2(dct_90) + 128);
im6 = uint8(idct2(dct_180) + 128);

% 显示结果
figure;
subplot(2,2,1); imshow(block); title('原始图像块');
subplot(2,2,2); imshow(im4); title('转置 DCT 还原');
```

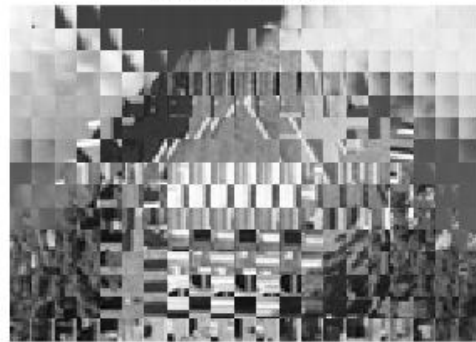
```
subplot(2,2,3); imshow(im5); title('旋转 90°的 DCT 还原');  
subplot(2,2,4); imshow(im6); title('旋转 180°的 DCT 还原');  
  
%% 函数声明  
function C = set_t(C)  
    C = C';  
end  
  
function C = set_90(C)  
    C = rot90(C);  
end  
  
function C = set_180(C)  
    C = rot90(C, 2);  
end
```

### (3) 运行结果

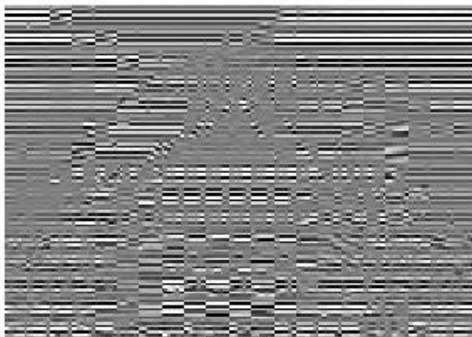
原始图片



转置DCT还原

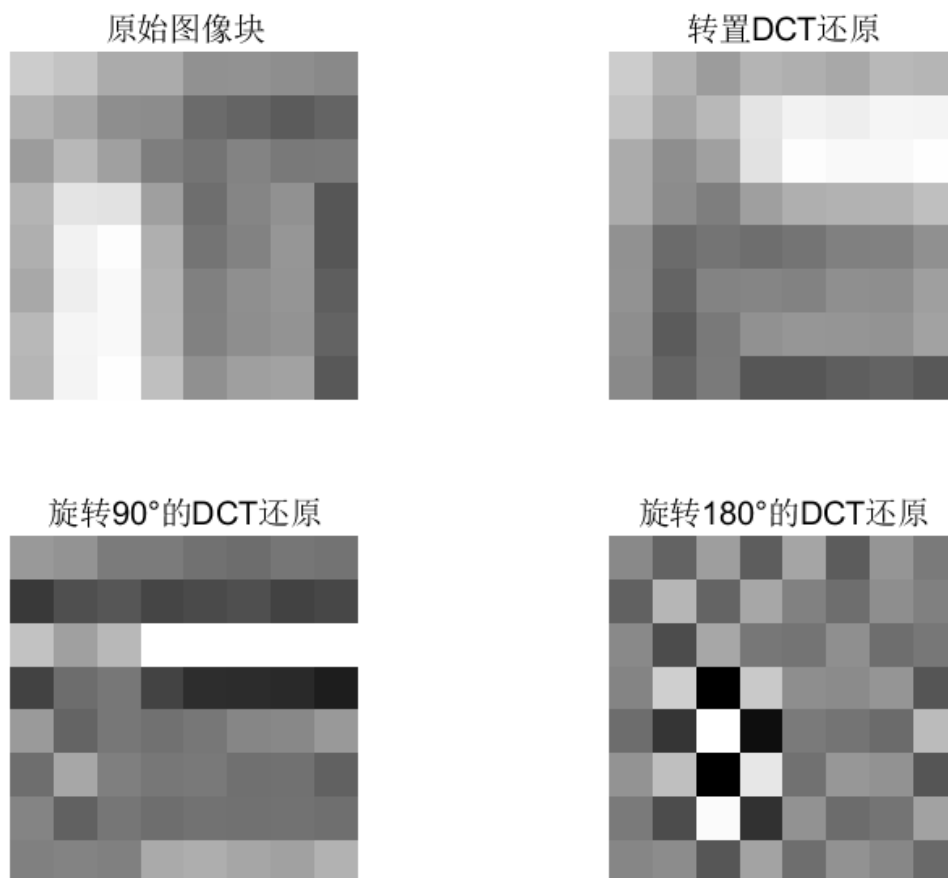


旋转90°的DCT还原



旋转180°的DCT还原





DCT 系数转置后再复原，因处理时的分块操作导致图像被分块转置，复原后的图像不是原图像的转置。但根据对  $8 \times 8$  子块的操作结果可知，每一小块使用 DCT 系数转置后再复原，恢复后的图像是原  $8 \times 8$  子块的转置。

DCT 系数旋转  $90^\circ$ ，左上角较大的直流分量，经变换移动至右上角，横向变化分量明显增加，所以复原后整张图片横向变化较大，出现横条纹。横条纹在完整图像的操作中更明显。

DCT 系数旋转  $180^\circ$ ，左上角较大的直流分量，经变换移动至右下角，横向变化分量和纵向变化分量均明显增加，所以复原后整张图片横向和纵向变化均较大，出现横纵相间的条纹。横条纹和纵条纹在完整图像的操作中更明显。

## 5. 差分编码系统的频率响应

### (1) 设计思路

根据指导书中对于差分编码的介绍，设置正确的系数，并调用 `freqz` 函数绘制频率响应图像。

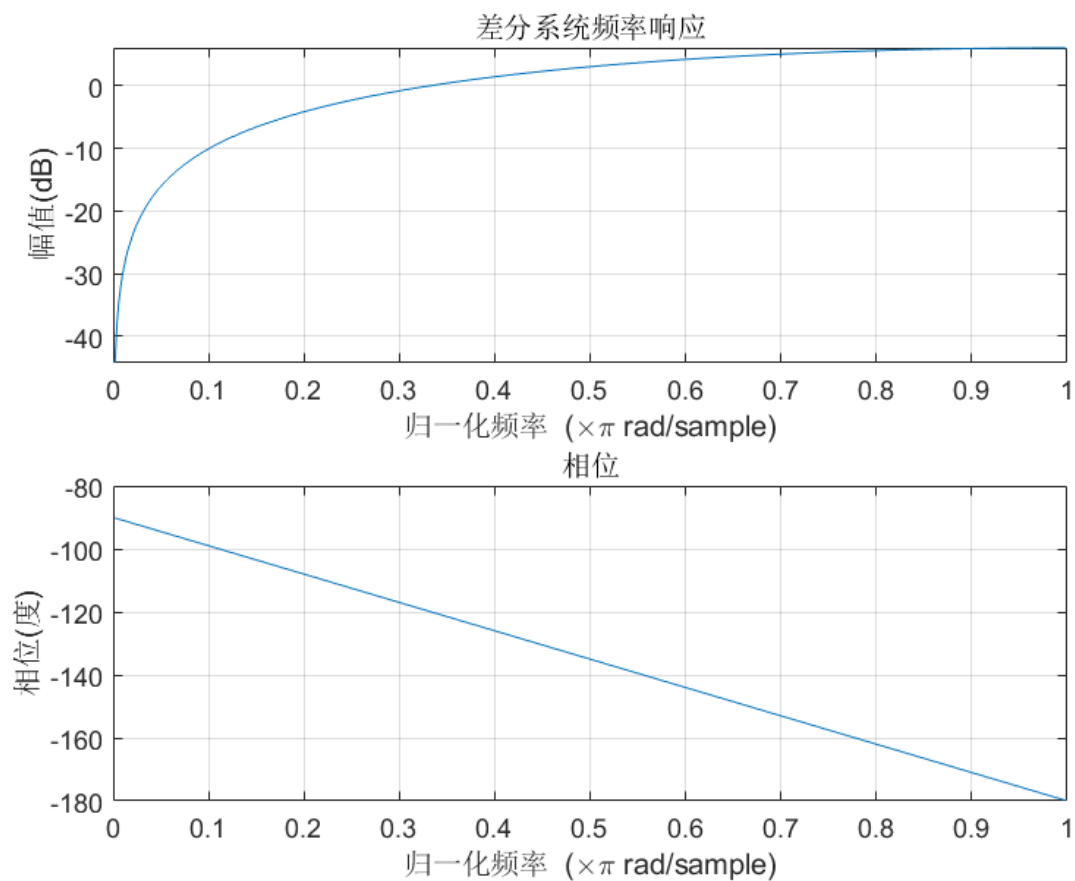
$$\hat{c}_D(n) = \begin{cases} \tilde{c}_D(n) & n = 1; \\ \tilde{c}_D(n-1) - \tilde{c}_D(n) & \text{elsewhere} \end{cases}$$

### (2) 核心代码

```
a = [1];  
b = [-1, 1];  
  
figure;  
freqz(b,a);  
title('差分系统频率响应');
```

### (3) 运行结果

差分编码系统是一个高通滤波器。DC 系数先进行差分编码再进行熵编码，说明 DC 系数高频分量更多。



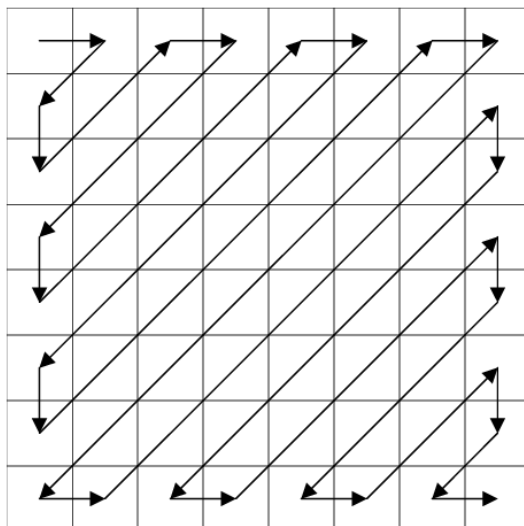
## 6. DC 预测误差的取值和 Category 的关系

Category 是 DC 预测误差的二进制表示的位数。利用 DC 预测误差可计算 Category， $\text{Category} = \text{ceil}(\log_2(\text{abs}(\text{DC\_error}) + 1))$ ;

## 7. Zig-Zag 扫描

### (1) 设计思路

Zig-Zag 扫描的方法包括预定义索引矩阵、向量化方法等。为实现对能适用于所有尺寸的通用 Zig-Zag 扫描函数，将扫描路径拆分为多条对角线的组合，并通过边界条件限制扫描范围，最终根据扫描顺序将对应位置的元素存储在结构矩阵中。最后，输出 Zig-Zag 扫描矩阵和矩阵每个位置的扫描顺序。



### (2) 核心代码

```
matrix = reshape(1:64, 8, 8)'; % 创建 8x8 测试矩阵
fprintf('原始 8x8 矩阵:\n');
disp(matrix);

result = zigzag(matrix); % 执行 Zig-Zag 扫描
fprintf('Zig-Zag 扫描结果:\n');
disp(result);

% 显示扫描顺序
fprintf('扫描顺序:\n');
order = zeros(8, 8);
for i = 1:64
    [row, col] = find(matrix == result(i));
    order(row, col) = i;
end
disp(order);

%% Zig-Zag 扫描函数
function result = zigzag(matrix)
```

```

[m, n] = size(matrix); % 获取矩阵尺寸
result = zeros(1, m * n); % 初始化结果矩阵

counter = 1; % 结果矩阵计数器
for s = 1:(m + n - 1) % 遍历所有对角线
    if mod(s, 2) == 0 % 偶数对角线，向下扫描
        for i = max(1, s - n + 1):min(s, m) % 行数的范围
            j = s - i + 1; % 计算对应元素的列数
            result(counter) = matrix(i, j);
            counter = counter + 1;
        end
    else % 奇数对角线，向上扫描
        for i = min(s, m):-1:max(1, s - n + 1) % 行数的范围
            j = s - i + 1; % 计算对应元素的列数
            result(counter) = matrix(i, j);
            counter = counter + 1;
        end
    end
end
end
end
end

```

### (3) 运行结果

由输出的 Zig-Zag 扫描结果可知，程序实现了预期扫描顺序。

```

>> HW2_2_7
原始8x8矩阵:
    1     2     3     4     5     6     7     8
    9    10    11    12    13    14    15    16
   17    18    19    20    21    22    23    24
   25    26    27    28    29    30    31    32
   33    34    35    36    37    38    39    40
   41    42    43    44    45    46    47    48
   49    50    51    52    53    54    55    56
   57    58    59    60    61    62    63    64

扫描顺序:
    1     2     6     7    15    16    28    29
    3     5     8    14    17    27    30    43
    4     9    13    18    26    31    42    44
   10    12    19    25    32    41    45    54
   11    20    24    33    40    46    53    55
   21    23    34    39    47    52    56    61
   22    35    38    48    51    57    60    62
   36    37    49    50    58    59    63    64

Zig-Zag扫描结果:
列 1 至 18
    1     2     9    17    10     3     4    11    18    25    33    26    19    12     5     6    13    20

列 19 至 36
   27    34    41    49    42    35    28    21    14     7     8    15    22    29    36    43    50    57

列 37 至 54
   58    51    44    37    30    23    16    24    31    38    45    52    59    60    53    46    39    32

列 55 至 64
   40    47    54    61    62    55    48    56    63    64

```

## 8. 对测试图像分块、DCT 和量化

### (1) 设计思路

需对测试图像进行分块、DCT、量化处理，再用 Zig-Zag 扫描形成列矢量后进行拼接，最终得到的整个系数矩阵每列为一个块的 DCT 系数，每行为各个块的 DC 系数。可复用上一小问的 Zig-Zag 扫描函数。

### (2) 核心代码

```
% 加载数据
load('hall.mat');
load('JpegCoeff.mat');

% 图像预处理
im = double(hall_gray) - 128;
[h, w] = size(im);

Co = trans(im, QTAB); % 求系数矩阵，参数是预处理后的图像和标准量化表

% 显示结果
fprintf('系数矩阵尺寸: %d x %d\n', size(Co));
fprintf('DC 系数: ');
disp(Co(1, :));

%% Zig-Zag 扫描函数
function result = zigzag(matrix)
    [m, n] = size(matrix); % 获取矩阵尺寸
    result = zeros(1, m * n); % 初始化结果矩阵

    counter = 1; % 结果矩阵计数器
    for s = 1:(m + n - 1) % 遍历所有对角线
        if mod(s, 2) == 0 % 偶数对角线，向下扫描
            for i = max(1, s - n + 1):min(s, m) % 行数的范围
                j = s - i + 1; % 计算对应元素的列数
                result(counter) = matrix(i, j);
                counter = counter + 1;
            end
        else % 奇数对角线，向上扫描
            for i = min(s, m):-1:max(1, s - n + 1) % 行数的范围
                j = s - i + 1; % 计算对应元素的列数
                result(counter) = matrix(i, j);
                counter = counter + 1;
            end
        end
    end
end
```



```

        end
    end
end

%% 系数矩阵计算函数
function Co = trans(im, QTAB)
    [h, w] = size(im);
    %分成多个 8*8 大小的子块
    wp = w / 8;
    hp = h / 8;

    Co = zeros(64, wp * hp); % 初始化系数矩阵
    idx = 1; % 初始化索引

    for i = 1 : 8: h % 遍历每一行
        for j = 1 : 8: w % 遍历每一列
            block = im(i : i + 7, j : j + 7); % 对图像分块
            dct = dct2(block); % DCT 变换
            quant = round(dct ./ QTAB); % 量化
            Zigzag = zigzag(quant); % Zig-Zag 扫描
            Co(:, idx) = Zigzag'; % 存储结果, zigzag 函数返回的是行向量, 需
要转置
            idx = idx + 1;
        end
    end
end
end

```

### (3) 运行结果

由输出的 Zig-Zag 扫描结果可知, 程序实现了预期扫描顺序。

```

>> HW2_2_8
系数矩阵尺寸: 64 x 315
DC系数: 列 1 至 19

```

```

    57    51     9   -28   -32   -32   -32   -32   -32   -32   -32   -25    27    40    45    50    53    55    55

列 20 至 38
    51    50    53    51    51    23    -5   -30   -31   -31   -29   -21   -16    -1    43    51    53    52    52

列 39 至 57
    53    53    54    52    48    35    34    32    53    17   -25   -21    -2     4    12    16    19    35    47

列 58 至 76
    43    47    49    51    53    53    33    34    26    22    40    39   -15   -26   -17   -10    -7    -5    -2

列 77 至 95
    11    18    -4    10    23    38    46    48    33    29    24    25    24   -20   -37   -35     2     6    10

```

列 96 至 114

4 11 27 23 -2 -23 -8 10 29 40 25 25 20 17 -7 -28 -37 -31 12

列 115 至 133

8 4 8 19 32 35 16 -3 1 -7 4 20 4 10 16 16 -38 -40 -15

列 134 至 152

2 6 7 6 7 3 18 33 23 18 11 -21 -20 -14 -18 -9 3 17 10

列 153 至 171

15 7 35 36 35 35 35 34 19 10 19 15 8 -18 -20 -19 -5 -4 -5

列 172 至 190

-11 -20 4 -7 38 40 39 36 36 36 19 1 -3 -4 -11 -12 -16 -18 -8

列 191 至 209

-10 -20 -37 -26 -6 -8 19 18 18 15 17 11 12 -2 2 14 -2 -17 -13

列 210 至 228

-16 -15 -6 -27 -27 -24 -3 -19 9 -22 13 -27 14 -29 16 5 -11 -25 -21

列 229 至 247

-29 2 -4 -16 -17 -27 -23 -12 14 -7 36 14 29 13 26 3 13 -27 -10

列 248 至 266

-20 -25 -28 -24 -18 -30 -27 -25 -32 -14 13 -10 26 -8 28 -18 31 -27 -7

列 267 至 285

-38 -38 -33 -30 -30 -22 -27 -27 -33 -39 -30 -27 9 -13 16 -22 19 -34 22

列 286 至 304

-43 -22 -38 -35 -18 -15 -32 -34 -38 -24 -34 -27 -30 -19 20 18 22 8 23

列 305 至 315

-3 19 -19 -27 -30 -27 11 1 -18 -28 -32

## 9. 实现 JPEG 编码

### (1) 设计思路

按照指导书，依次进行分块，DCT 变换，量化求得系数矩阵后，分别提取 DC 系数和 AC 系数并进行编码。DC 系数编码方法为 DPCM 差分编码+Huffman 编码，AC 系数编码方法为游程编码+Huffman 编码。将主要的 JPEG 编码操作封装在 JPEG\_encode 函数中，方便后续调用。

### (2) 核心代码

```
load('hall.mat');  
load('JpegCoeff.mat');  
  
[DC_ans, AC_ans, h, w] = JPEG_encode(hall_gray, QTAB, DCTAB, ACTAB);
```

```
save('jpegcodes.mat', 'DC_ans', 'AC_ans', 'h', 'w'); % 保存结果到
jpegcodes.mat

% 计算并显示
comp = w * h * 8 / (length(DC_ans) + length(AC_ans)); % 压缩比 = 输入文件
长度（像素数 * 每个像素 8bit） / 输出码流长度
fprintf('图像尺寸: %d x %d\n', h, w);
fprintf('DC 码流长度: %d bits\n', length(DC_ans));
fprintf('AC 码流长度: %d bits\n', length(AC_ans));
fprintf('总码流长度: %d bits\n', length(DC_ans) + length(AC_ans));
fprintf('压缩比: %.3f:1\n', comp);

function [DC_ans, AC_ans, h, w] = JPEG_encode(image_ori, QTAB, DCTAB,
ACTAB)
    [h, w] = size(image_ori);
    im = double(image_ori) - 128;

    Co = trans(im, QTAB); % 求系数矩阵，参数是预处理后的图像和标准量化表

    % DC 系数处理
    DC = Co(1, :); % DC 系数是系数矩阵的第一行
    DC_ans = DCEncode(DC, DCTAB); % DPCM 差分编码 + Huffman 编码

    % AC 系数处理
    AC = Co(2:end, :); % AC 系数是系数矩阵除了第一行的部分
    AC_ans = [];
    n = w * h / 64; % 块数

    for idx = 1 : n % 遍历所有子块
        AC_idx = AC(:, idx)'; % 得到当前块的所有 AC 系数
        AC_block = ACEncode(AC_idx, ACTAB); % 游程编码 + Huffman 编码
        AC_ans = [AC_ans, AC_block]; % 合并 AC 熵编码
    end
end

%% 系数矩阵计算函数
function Co = trans(im, QTAB)
    [h, w] = size(im);
    %分成多个 8*8 大小的子块
    wp = w / 8;
    hp = h / 8;

    Co = zeros(64, wp * hp); % 初始化系数矩阵
    idx = 1; % 初始化索引
```

```
for i = 1 : 8: h % 遍历每一行
    for j = 1 : 8: w % 遍历每一列
        block = im(i : i + 7, j : j + 7); % 对图像分块
        dct = dct2(block); % DCT 变换
        quant = round(dct ./ QTAB); % 量化
        Zigzag = zigzag(quant); % Zig-Zag 扫描
        Co(:, idx) = Zigzag'; % 存储结果, zigzag 函数返回的是行向量, 需
要转置
        idx = idx + 1;
    end
end
end

%% Zig-Zag 扫描函数
function result = zigzag(matrix)
    [m, n] = size(matrix); % 获取矩阵尺寸
    result = zeros(1, m * n); % 初始化结果矩阵

    counter = 1; % 结果矩阵计数器
    for s = 1:(m + n - 1) % 遍历所有对角线
        if mod(s, 2) == 0 % 偶数对角线, 向下扫描
            for i = max(1, s - n + 1):min(s, m) % 行数的范围
                j = s - i + 1; % 计算对应元素的列数
                result(counter) = matrix(i, j);
                counter = counter + 1;
            end
        else % 奇数对角线, 向上扫描
            for i = min(s, m):-1:max(1, s - n + 1) % 行数的范围
                j = s - i + 1; % 计算对应元素的列数
                result(counter) = matrix(i, j);
                counter = counter + 1;
            end
        end
    end
end

%% DC 系数编码函数
function DC_ans = DCencode(DC, DCTAB)
    DC_ans = [];
    DC = [DC(1), -diff(DC)]; % DPCM 差分编码

    for block = 1:length(DC)
        mag = DC(block); % 计算 Magnitude
```

```
% 计算 Category
if mag == 0
    cat = 0;
else
    cat = ceil(log2(abs(mag) + 1)); % Category 计算公式
end

% 获取 Huffman 码字
code_length = DCTAB(cat + 1, 1); % 获取码字长度
huffman = DCTAB(cat + 1, 2:code_length + 1); % 获取对应码字内容

% 幅度值二进制编码
if mag > 0
    mag_bin = double(dec2bin(mag, cat)) - 48; % dec2bin 将正数转换为指定位数的二进制字符串, -48 将字符转换为数字
elseif mag < 0
    mag_bin = double(dec2bin(2^cat + mag - 1, cat)) - 48; % 负数用补码表示
else
    mag_bin = [];
end

DC_ans = [DC_ans, huffman, mag_bin]; % 组合 DC 熵编码
end

end

%% AC 系数编码函数
function AC_ans = ACencode(AC_idx, ACTAB)
    AC_ans = [];
    nzero_pos = find(AC_idx); % 找到所有非零系数的位置

    if isempty(nzero_pos)
        AC_ans = [1, 0, 1, 0]; % 如果全为 0, 直接使用 EOB 结束符并返回
        return;
    end

    % 计算零游程长度
    if nzero_pos(1) == 1 % 第一个系数非零
        run_length = [0, diff(nzero_pos) - 1]; % 相邻非零系数位置的差值减去当前非零系数本身的位置即为 0 的个数
    else
        run_length = [nzero_pos(1) - 1, diff(nzero_pos) - 1];
    end
end
```

```
% 处理每个非零系数
for idx = 1:length(nzero_pos)
    run = run_length(idx);
    amp = AC_idx(nzero_pos(idx));

    % 处理长零游程
    while run > 15
        AC_ans = [AC_ans, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1]; % 插入
ZRL 符号
        run = run - 16;
    end

    % 计算 size
    if amp == 0
        size = 0;
    else
        size = ceil(log2(abs(amp) + 1)); % Size 和 DC 编码的 Category 计
算公式相同
    end

    % 获取 Huffman 码字
    index = find(ACTAB(:, 1) == run & ACTAB(:, 2) == size, 1); %
Run/Size
    code_length = ACTAB(index, 3); % 获取码字长度
    huffman = ACTAB(index, 4:code_length + 3); % 获取对应码字内容

    % 幅度值二进制编码，类似 DC 编码过程
    if amp > 0
        amp_bin = double(dec2bin(amp, size)) - 48; % dec2bin 将正数转
换为指定位数的二进制字符串，-48 将字符串转换为数字
    elseif amp < 0
        amp_bin = double(dec2bin(2^size + amp - 1, size)) - 48; % 负
数用补码表示
    else
        amp_bin = [];
    end

    AC_ans = [AC_ans, huffman, amp_bin]; % 组合 AC 熵编码
end

AC_ans = [AC_ans, 1, 0, 1, 0]; % 插入块结束符 EOB
end
```

### (3) 运行结果

程序可正常运行，输出图像尺寸、DC 码流长度、AC 码流长度、总码流长度和压缩比。

```
>> HW2_2_9  
图像尺寸: 120 x 168  
DC码流长度: 2031 bits  
AC码流长度: 23072 bits  
总码流长度: 25103 bits  
压缩比: 6.425:1
```

## 10. 计算 JPEG 编码的压缩比

### (1) 设计思路

利用压缩比计算公式:  $\text{压缩比} = \text{输入文件长度 (像素数} \times \text{每个像素 8bit)} / \text{输出码流长度}$ ，需转换为相同进制。

### (2) 核心代码

同上一小问，仅摘取核心计算部分。

```
comp = w * h * 8 / (length(DC_ans) + length(AC_ans)); % 压缩比 = 输入文件  
长度 (像素数 * 每个像素 8bit) / 输出码流长度
```

### (3) 运行结果

计算出压缩比为 6.425:1。

```
>> HW2_2_9  
图像尺寸: 120 x 168  
DC码流长度: 2031 bits  
AC码流长度: 23072 bits  
总码流长度: 25103 bits  
压缩比: 6.425:1
```

## 11. 实现 JPEG 编码并评价性能

### (1) 设计思路

按照指导书，依次进行编码的逆向操作。先对 DC 和 AC 系数分别进行解码，得到系数

矩阵。然后进行逆向量化，逆 DCT 变化，将每一块放置在正确位置，拼回完整图像后再进行逆向预处理操作。将主要的 JPEG 解码操作封装在 JPEG\_decode 函数中，方便后续调用。在解码完成后，从客观和主观两个维度共同评价图像质量：计算峰值信噪比 PSNR，客观评价图像质量；仔细观察解码后的图像与原图像，对图像质量进行主观评价。

## (2) 核心代码

```
%% 编码过程
load('hall.mat');
load('JpegCoeff.mat');

[DC_ans, AC_ans, h, w] = JPEG_encode(hall_gray, QTAB, DCTAB, ACTAB);

save('jpegcodes.mat', 'DC_ans', 'AC_ans', 'h', 'w'); % 保存结果到
jpegcodes.mat

% 计算并显示
comp = w * h * 8 / (length(DC_ans) + length(AC_ans)); % 压缩比 = 输入文件
长度（像素数 * 每个像素 8bit） / 输出码流长度
fprintf('图像尺寸: %d x %d\n', h, w);
fprintf('DC 码流长度: %d bits\n', length(DC_ans));
fprintf('AC 码流长度: %d bits\n', length(AC_ans));
fprintf('总码流长度: %d bits\n', length(DC_ans) + length(AC_ans));
fprintf('压缩比: %.3f:1\n', comp);

%% 解码过程
load('jpegcodes.mat');

image = JPEG_decode(DC_ans, AC_ans, h, w, QTAB, DCTAB, ACTAB);

% 计算并显示
MSE = sum(sum((double(image) - double(hall_gray)).^2)) / (h * w);
PSNR = 10 * log10(255 ^ 2 / MSE); % 计算 PSNR
fprintf('PSNR: %.3f dB\n', PSNR);

figure; imshow(hall_gray, 'InitialMagnification', 'fit'); title('原始图
像');
figure; imshow(image, 'InitialMagnification', 'fit'); title('解码图像');

function image = JPEG_decode(DC_ans, AC_ans, h, w, QTAB, DCTAB, ACTAB)
    % 初始化参数
    block = h * w / 64;
    wb = w / 8;
```



```
hb = h / 8;
image = zeros(h, w);

DC_ans1 = DCdecode(DC_ans, DCTAB, block); % DC 系数解码
AC_ans1 = ACdecode(AC_ans, ACTAB, block); % AC 系数解码
im_ans1 = cat(1, DC_ans1, AC_ans1); % 合并 DC 和 AC 系数

for i = 1 : hb
    for j = 1 : wb
        Co = im_ans1(:, (i - 1) * wb + j); % 获取当前块的系数向量
        im = zigzag_i(Co, 8, 8); % 使用反向 zigzag 扫描将向量转换为 8x8
        im = im .* QTAB; % 逆向量化
        im = idct2(im); % 逆 DCT 变换
        image((8 * i - 7):8 * i, (8 * j - 7):8 * j) = im; % 将块放置
    end
end
image = uint8(image + 128); % 逆向预处理操作
end

%% 逆向 Zig-Zag 扫描函数
function matrix = zigzag_i(vector, m, n)
    matrix = zeros(m, n);
    counter = 1;

    for s = 1:(m + n - 1)
        if mod(s, 2) == 0 % 偶数对角线，向下扫描
            for i = max(1, s - n + 1) : min(s, m)
                j = s - i + 1;
                matrix(i, j) = vector(counter);
                counter = counter + 1;
            end
        else % 奇数对角线，向上扫描
            for i = min(s, m) : -1 : max(1, s - n + 1)
                j = s - i + 1;
                matrix(i, j) = vector(counter);
                counter = counter + 1;
            end
        end
    end
end

%% DC 解码函数
```

```

function DC_ans1 = DCdecode(DC_ans, DCTAB, block)
    DC_ans1 = zeros(1, block); % 创建空矩阵存储 DC 系数
    count = 1; % 当前块位置
    i = 1; % 编码位置

    while i <= size(DC_ans, 2) && count <= block
        for j = 1:size(DCTAB, 1)
            code_length = DCTAB(j, 1); % Huffman 编码长度
            if i + code_length - 1 <= size(DC_ans, 2) && all(DCTAB(j,
2 : 1 + code_length) == DC_ans(1, i : i + code_length - 1))
                i = i + code_length; % 跳过 Huffman 编码
                cat = j - 1; % 计算幅度值类别
                if cat == 0
                    DC_ans1(1, count) = 0;
                else
                    if i + cat - 1 <= size(DC_ans, 2)
                        mag = DC_ans(1, i : i + cat - 1); % 读取幅度值比特
                        % 解码幅度值
                        if mag(1) == 1
                            DC_ans1(1, count) = bin2dec(char(mag +
'0'))); % 正数直接转十进制
                        else
                            DC_ans1(1, count) = -bin2dec(char(~mag +
'0'))); % 负数先取反转十进制再加负号
                        end
                        i = i + cat;
                    end
                end
                count = count + 1;
                break;
            end
        end
        end

        for n = 2 : block
            DC_ans1(1, n) = DC_ans1(1, n - 1) - DC_ans1(1, n); % 差分解码
        end
    end

end

%% AC 解码函数
function AC_ans1 = ACdecode(AC_ans, ACTAB, block)
    AC_ans1 = zeros(block, 63); % 创建空矩阵存储 63 个 AC 系数
    block = 1; % 当前处理的块编号
    count = 1; % 当前块内 AC 系数位置

```

```

i = 1; % 编码位置
ZRL = [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1];
EOB = [1, 0, 1, 0];

while i <= size(AC_ans, 2) % 编码位置不超过 AC 编码长度时
    if i + length(EOB) - 1 <= size(AC_ans, 2) && all(EOB ==
AC_ans(1, i:i+length(EOB)-1)) % 遇到 EOB 表示当前块的所有 AC 系数都已解码完毕
        count = 1; % 重置系数位置
        i = i + length(EOB); % 移动编码位置
        block = block + 1; % 移动到下一个块

    elseif i + length(ZRL) - 1 <= size(AC_ans, 2) && all(ZRL ==
AC_ans(1, i:i+length(ZRL)-1)) % 遇到 ZRL 直接填充 16 个零，避免逐个解码
        AC_ans1(block, count:count+15) = 0; % 直接填充 16 个 0
        count = count + 16; % 移动系数位置
        i = i + length(ZRL); % 移动编码位置

    else % 一般情况
        for j = 1 : size(ACTAB, 1)
            code_length = ACTAB(j, 3); % Huffman 编码长度
            if i + code_length - 1 <= size(AC_ans, 2) && all(ACTAB(j,
4 : 3 + code_length) == AC_ans(1, i : i + code_length - 1))
                i = i + code_length; % 跳过 Huffman 编码
                run = ACTAB(j, 1); % 零游程长度
                size_bit = ACTAB(j, 2); % 幅度值比特数

                % 添加零游程
                AC_ans1(block, count : count + run) = 0;
                count = count + run;

                % 解码幅度值
                if size_bit > 0
                    if i + size_bit - 1 <= size(AC_ans, 2)
                        amplitude = AC_ans(1, i : i + size_bit - 1);
                        if amplitude(1) == 1
                            AC_ans1(block, count) =
bin2dec(char(amplitude + '0')); % 正数直接转十进制
                        else
                            AC_ans1(block, count) = -
bin2dec(char(~amplitude + '0')); % 负数先取反转十进制再加负号
                        end
                        count = count + 1;
                        i = i + size_bit;
                    end
                end
            end
        end
    end
end

```

```
        end
        break;
    end
end
end
end
AC_ans1 = AC_ans1';
end
```

### (3) 运行结果

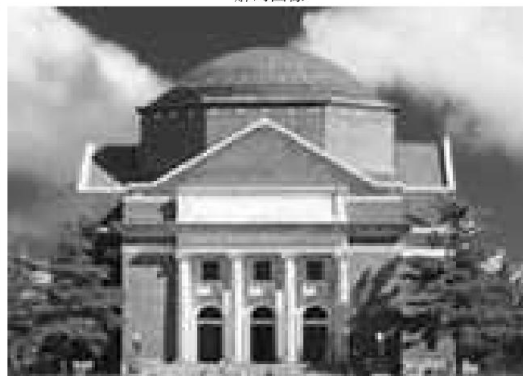
计算出 PSNR 为 31.187dB，数值较大，说明图像质量较好，失真较小。主观上，解码后的图像相较于原始图像，能看到分块拼接的边界，部分建筑物边缘细节丢失，且整体更加模糊。总体而言，说明 JPEG 编码解码后生成的图像整体质量较好，部分细节丢失但不太影响基本观感，也验证了代码的可行性。

```
>> HW2_2_11
图像尺寸: 120 x 168
DC码流长度: 2031 bits
AC码流长度: 23072 bits
总码流长度: 25103 bits
压缩比: 6.425:1
PSNR: 31.187 dB
```

原始图像



解码图像



## 12. 将量化步长减小到一半重新编码解码

### (1) 设计思路

因已将编码和解码的功能封装在函数中，只需在调用时将传入的量化表参量 QTAB 改为 QTAB/2，即可实现将量化步长减小到一半。重新进行 JPEG 编码和解码操作，计算压缩比、PSNR 等指标，并输出编码解码后的复原图像。

## (2) 核心代码

```
%% 编码过程
load('hall.mat');
load('JpegCoeff.mat');

[DC_ans, AC_ans, h, w] = JPEG_encode(hall_gray, QTAB / 2, DCTAB,
ACTAB);

save('jpegcodes1.mat', 'DC_ans', 'AC_ans', 'h', 'w'); % 保存结果到
jpegcodes1.mat

% 计算并显示
comp = w * h * 8 / (length(DC_ans) + length(AC_ans)); % 压缩比 = 输入文件
长度（像素数 * 每个像素 8bit） / 输出码流长度
fprintf('图像尺寸: %d x %d\n', h, w);
fprintf('DC 码流长度: %d bits\n', length(DC_ans));
fprintf('AC 码流长度: %d bits\n', length(AC_ans));
fprintf('总码流长度: %d bits\n', length(DC_ans) + length(AC_ans));
fprintf('压缩比: %.3f:1\n', comp);

%% 解码过程
load('jpegcodes1.mat');

image = JPEG_decode(DC_ans, AC_ans, h, w, QTAB / 2, DCTAB, ACTAB);

% 计算并显示
MSE = sum(sum((double(image) - double(hall_gray)).^2)) / (h * w);
PSNR = 10 * log10(255^2 / MSE); % 计算 PSNR
fprintf('PSNR: %.3f dB\n', PSNR);

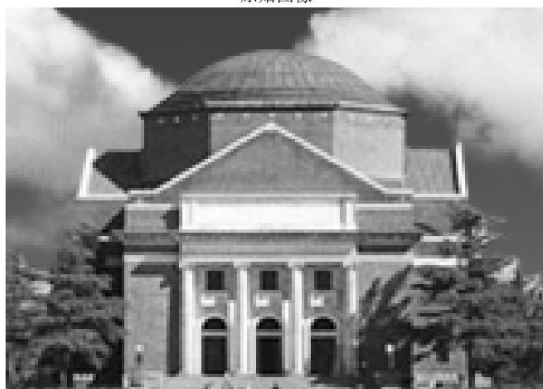
figure; imshow(hall_gray, 'InitialMagnification', 'fit'); title('原始图
像');
figure; imshow(image, 'InitialMagnification', 'fit'); title('解码图像');
```

## (3) 运行结果

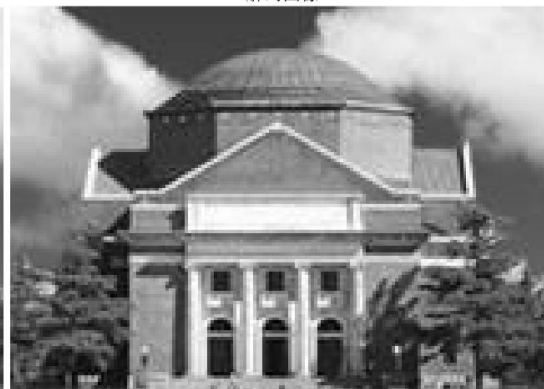
计算出压缩比为 4.410:1, PSNR 为 34.207dB。对比之前的结果,发现量化步长减小到一半后,压缩比变小,PSNR 变大——即图像质量提升,失真变小,但压缩比变小导致图像需要的储存空间增大。而根据主观观察,编码解码复原后的图像在整体和细节上都变得更加清晰,说明减小量化步长使得更多的高频分量被保留。

```
>> HW2_2_12
图像尺寸: 120 x 168
DC码流长度: 2410 bits
AC码流长度: 34164 bits
总码流长度: 36574 bits
压缩比: 4.410:1
PSNR: 34.207 dB
```

原始图像



解码图像



### 13. 对雪花图像重新编码解码

#### (1) 设计思路

因已将编码和解码的功能封装在函数中，只需在调用时将传入的图像参量 `hall_gray` 改为 `snow`，即可实现对雪花图像重新进行 JPEG 编码和解码操作。计算压缩比、PSNR 等指标，并输出编码解码后的复原图像。

#### (2) 核心代码

```
%% 编码过程
load('snow.mat');
load('JpegCoeff.mat');

[DC_ans, AC_ans, h, w] = JPEG_encode(snow, QTAB, DCTAB, ACTAB);

save('jpegcodes_snow.mat', 'DC_ans', 'AC_ans', 'h', 'w'); % 保存结果到
jpegcodes_snow.mat

% 计算并显示
comp = w * h * 8 / (length(DC_ans) + length(AC_ans)); % 压缩比 = 输入文件
长度（像素数 * 每个像素 8bit） / 输出码流长度
fprintf('图像尺寸: %d x %d\n', h, w);
fprintf('DC 码流长度: %d bits\n', length(DC_ans));
```

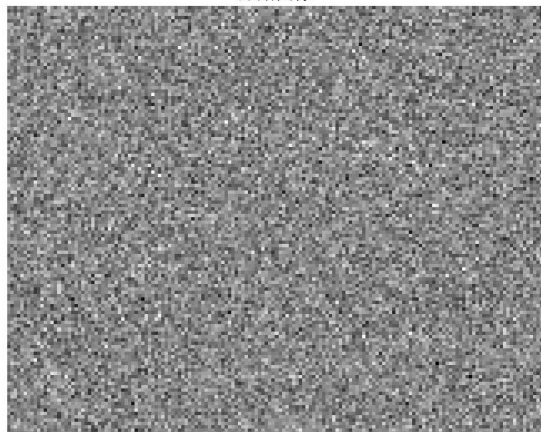
```
fprintf('AC 码流长度: %d bits\n', length(AC_ans));  
fprintf('总码流长度: %d bits\n', length(DC_ans) + length(AC_ans));  
fprintf('压缩比: %.3f:1\n', comp);  
  
%% 解码过程  
load('jpegcodes_snow.mat');  
  
image = JPEG_decode(DC_ans, AC_ans, h, w, QTAB, DCTAB, ACTAB);  
  
% 计算并显示  
MSE = sum(sum((double(image) - double(snow)).^2)) / (h * w);  
PSNR = 10 * log10(255^2 / MSE); % 计算 PSNR  
fprintf('PSNR: %.3f dB\n', PSNR);  
  
figure; imshow(snow, 'InitialMagnification', 'fit'); title('原始图像');  
figure; imshow(image, 'InitialMagnification', 'fit'); title('解码图像');
```

### (3) 运行结果

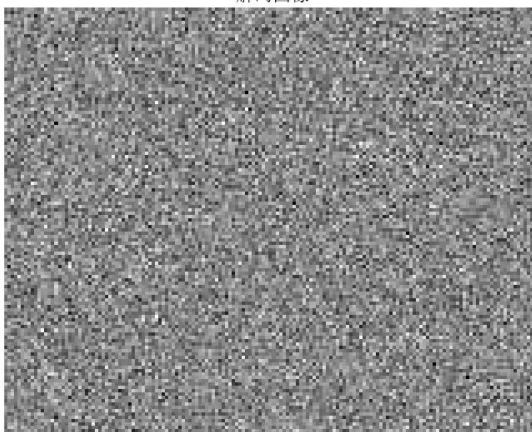
计算出压缩比为 3.645:1, PSNR 为 22.924dB。与大礼堂图片相比,发现雪花图片的压缩比和 PSNR 均变小——即雪花图片经过 JPEG 编码和解码,图像质量降低得更多,失真较大,可能原因是雪花图像有更多的高频分量。

```
>> HW2_2_13  
图像尺寸: 128 x 160  
DC码流长度: 1403 bits  
AC码流长度: 43546 bits  
总码流长度: 44949 bits  
压缩比: 3.645:1  
PSNR: 22.924 dB
```

原始图像



解码图像





## 四、 信息隐藏

### 1. 实现空域隐藏和提取方法

#### (1) 设计思路

隐藏信息为生成的随机比特流，利用 LSB 替换法将信息嵌入图像空域中，并验证提取正确率。随后对含密图像进行完整的 JPEG 编码解码处理，再次提取信息并计算正确率，同时通过 PSNR 指标量化图像质量损失。通过 10 次独立实验获取统计平均值和标准差，最终以可视化图表形式呈现 JPEG 编码前后的正确率变化及 PSNR 变化情况，以获得空域 LSB 隐藏方法的抗 JPEG 编码能力。

#### (2) 核心代码

```
load('hall.mat');
load('JpegCoeff.mat');

n = 10; % 试验次数
msg_length = 1000; % 测试 1000 个比特
fprintf('试验次数: %d\n', n);

% 初始化统计变量
acc1 = zeros(1, n); % JPEG 编码前正确率
acc2 = zeros(1, n); % JPEG 编码后正确率
PSNR = zeros(1, n); % PSNR

for i = 1 : n
    msg_bits = randi([0, 1], 1, msg_length); % 生成随机测试信息
    row0 = 50; col0 = 50; % 隐藏信息的初始行列
    image_LSB = LSB_hide(hall_gray, msg_bits, row0, col0); % 空域信息隐藏 (LSB 替换)
    extracted_bits = LSB_seek(image_LSB, msg_length, row0, col0); % 从含密图像中提取信息

    % 计算未进行 JPEG 编码时的正确率
    err1 = sum(extracted_bits ~= msg_bits) / msg_length * 100;
    acc1(i) = 100 - err1;

    % 对图像进行 JPEG 编码和解码
    [DC_ans, AC_ans, h, w] = JPEG_encode(image_LSB, QTAB, DCTAB, ACTAB);
    image_JPEG = JPEG_decode(DC_ans, AC_ans, h, w, QTAB, DCTAB, ACTAB);

    % 计算 PSNR
```



```
MSE = sum(sum((double(image_JPEG) - double(image_LSB)).^2)) / (h * w);
PSNR(i) = 10 * log10(255^2 / MSE);

msg_bits1 = LSB_seek(image_JPEG, msg_length, row0, col0); % 从 JPEG 编码后的图像中提取信息

% 计算正确率
err2 = sum(msg_bits1 ~= msg_bits) / msg_length * 100;
acc2(i) = 100 - err2;
end

% 计算平均值和标准差
acc1_mean = mean(acc1);
acc1_std = std(acc1);
acc2_mean = mean(acc2);
acc2_std = std(acc2);
PSNR_mean = mean(PSNR);
PSNR_std = std(PSNR);

% 显示结果
fprintf('未编码时平均正确率: %.2f%% ± %.2f%%\n', acc1_mean, acc1_std);
fprintf('JPEG 编码后平均正确率: %.2f%% ± %.2f%%\n', acc2_mean, acc2_std);
fprintf('平均 PSNR: %.2f dB ± %.2f dB\n', PSNR_mean, PSNR_std);

figure;
subplot(2,1,1);
plot(1:n, acc1, 'bo-', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot(1:n, acc2, 'ro-', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
title('正确率');
xlabel('试验次数');
ylabel('正确率 (%)');
legend('JPEG 编码前', 'JPEG 编码后');
grid on;

subplot(2,1,2);
plot(1:n, PSNR, 'go-', 'LineWidth', 1.5, 'MarkerFaceColor', 'g');
title('PSNR');
xlabel('试验次数');
ylabel('PSNR (dB)');
grid on;

%% 辅助函数定义
```

```
function image_LSB = LSB_hide(image, msg_bits, row0, col0)
    image_LSB = image;
    [h, w] = size(image); % 获取图像尺寸
    msg_idx = 1; % 初始化信息索引

    for row = row0 : h % 遍历图像的每行每列
        for col = col0 : w
            if msg_idx > length(msg_bits) % 如果所有信息都已隐藏，则提前返回
                return;
            end
            image_LSB(row, col) = bitset(image_LSB(row, col), 1, msg_bits(msg_idx)); % 修改像素的 LSB 用于隐藏信息
            msg_idx = msg_idx + 1; % 移动到下一个要隐藏的信息比特
        end
    end
end

function bits = LSB_seek(image_JPEG, msg_length, row0, col0)
    bits = zeros(1, msg_length);
    [h, w] = size(image_JPEG); % 获取图像尺寸
    msg_idx = 1; % 初始化信息索引

    for row = row0 : h % 遍历图像的每行每列
        for col = col0 : w
            if msg_idx > msg_length % 如果所有信息都已找到，则提前返回
                return;
            end
            bits(msg_idx) = bitget(image_JPEG(row, col), 1); % 读取像素的最低位
            msg_idx = msg_idx + 1; % 移动到下一个存储位置
        end
    end
end
```

### (3) 运行结果

未进行 JPEG 编码，直接提取信息时，正确率始终为 100%；而从 JPEG 编码解码后生成的复原图像提取隐藏信息，平均正确率不到 50%，说明在空域上隐藏信息基本不具备抗 JPEG 编码的能力。

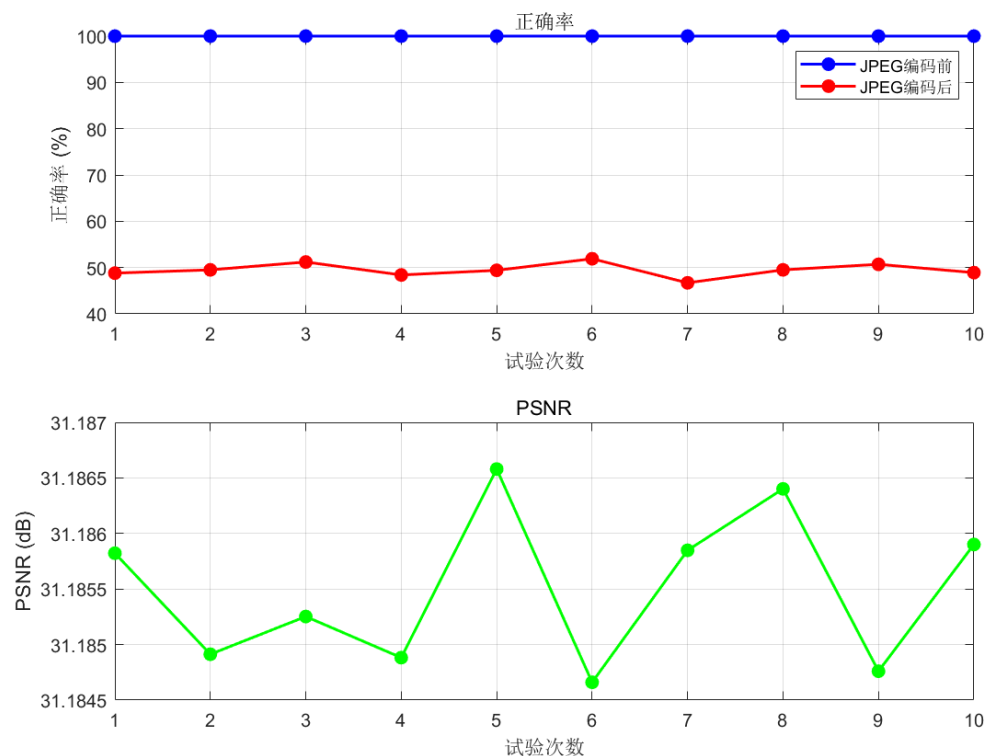
```
>> HW2_3_1
```

试验次数: 10

未编码时平均正确率: 100.00%  $\pm$  0.00%

JPEG编码后平均正确率: 49.50%  $\pm$  1.49%

平均PSNR: 31.18 dB  $\pm$  0.00 dB



## 2. 实现 3 种变换域信息隐藏和提取方法

### (1) 设计思路

**方法一:** 首先对原始图像进行分块, DCT 变换和量化, 然后将生成的随机信息比特逐一替换量化后 DCT 系数的 LSB 位。通过完整的 JPEG 编码流程生成压缩数据, 再执行解码过程并从中提取隐藏信息。最后计算压缩比、信息提取正确率和 PSNR, 并显示原始图像和含密图像。

**方法二:** 整体框架与方法一相同, 只是在信息隐藏和提取时, 只替换部分系数的 LSB, 嵌入信息的系数位置随机生成, 在每个块中随机选择 20 个位置 (除去 DC 系数) 进行信息隐藏。为便于信息提取, 需另用 pos 数组记录隐藏了信息的位置。

**方法三:** 整体框架与方法一、方法二相同, 只在信息隐藏和提取方法上略有区别。先将待隐藏信息用 1, -1 的序列表示, 再逐一将信息位追加在每个块 Zig-Zag 顺序的最后一个非零 DCT 系数之后; 如果原本该图像块的最后一个系数就不为零, 就用信息位替换该系数。

## (2) 核心代码

### 方法一

```
load ('hall.mat');
load ('JpegCoeff.mat');

[h, w] = size(hall_gray);
im = double(hall_gray) - 128; % 图像预处理

Co = trans(im, QTAB); % 分块, DCT 变换, 量化

info = double(randi([0, 1], 1000, 1)); % 生成 1000 比特的随机信息
pad = zeros(h * w - 1000, 1); % 填充剩余位置
info1 = cat(1, info, pad); % 合并成能覆盖图像所有位置的信息

Co_bin = dec2bin(Co); % 把系数转为二进制
Co_bin(:, end) = dec2bin(info1); % 在 LSB 存储隐藏信息
Co = zeros(size(Co));
for idx = 1 : numel(Co)
    if Co_bin(idx, 1) == '1' % 负数
        Co(idx) = bin2dec(Co_bin(idx, :)) - 256; % 补码处理
    else % 正数
        Co(idx) = bin2dec(Co_bin(idx, :));
    end
end

DC = Co(1, :); % 在系数矩阵中提取 DC 系数
DC_ans1 = DCencode(DC, DCTAB); % 对 DC 系数编码

AC = Co(2 : end, :); % 在系数矩阵中提取 AC 系数
AC_ans1 = [];
for idx = 1 : w * h / 64
    AC_idx = AC(:, idx)';
    AC_block = ACencode(AC_idx, ACTAB); % 对 AC 系数编码
    AC_ans1 = [AC_ans1 AC_block]; % 拼合所有块的 AC 系数
end

comp = h * w * 8 / (length(DC_ans1) + length(AC_ans1)); % 计算压缩比
fprintf("压缩比为%.3f\n", comp);

image = zeros(h, w);
block = h * w / 64;

DC_ans2 = DCdecode(DC_ans1, DCTAB, block); % 对 DC 系数解码
```

```
AC_ans2 = ACdecode(AC_ans1, ACTAB, block); % 对 AC 系数解码
im_ans2 = cat(1, DC_ans2, AC_ans2);

info2 = dec2bin(im_ans2); % 转换为二进制方便提取
info2 = bin2dec(info2(:, end)); % 提取隐藏在 LSB 的信息
acc = length(find(info == info2(1:1000))) / 1000; % 计算信息提取准确率
fprintf("信息恢复准确率为%.2f%%\n", acc * 100);

wb = w / 8;
hb = h / 8;

for i = 1 : hb
    for j = 1 : wb
        Co = im_ans2(:, (i - 1) * wb + j)'; % 获取当前块的系数向量
        im = zigzag_i(Co, 8, 8); % 使用反向 zigzag 扫描将向量转换为 8x8 矩阵

        im = im .* QTAB; % 逆向量化
        im = idct2(im); % 逆 DCT 变换
        image((8 * i - 7) : 8 * i, (8 * j - 7) : 8 * j) = im; % 将块放置
        % 在完整图像中的正确位置
    end
end
image = uint8(image + 128); % 逆向预处理操作

% 计算 PSNR
MSE = sum(sum((double(image) - double(hall_gray)).^2)) / (w * h);
PSNR = 10 * log10(255^2 / MSE);

fprintf("PSNR 为%.3f\n", PSNR);

figure; imshow(hall_gray, 'InitialMagnification', 'fit'); title('原图');
figure; imshow(image, 'InitialMagnification', 'fit'); title('隐藏信息图');
');
```

## 方法二

```
load ('hall.mat');
load ('JpegCoeff.mat');

[h, w] = size(hall_gray);
im = double(hall_gray) - 128; % 图像预处理

Co = trans(im, QTAB); % 分块, DCT 变换, 量化
```

```
info = double(randi([0, 1], 1000, 1)); % 生成 1000 比特的随机信息

% 选择要嵌入信息的系数位置（不是所有系数都嵌入）
num = 20; % 每个块中嵌入信息的系数数量
block = w * h / 64;
pos = zeros(num, block);

for idx = 1 : block
    pos_idx = randperm(63, num) + 1; % 在每个块中随机选择 num 个位置，+1 跳过 DC 系数
    pos(:, idx) = pos_idx;
end

% 将信息嵌入到选定的系数位置
info_idx = 1; % 信息索引
Co_bin = dec2bin(Co); % 把系数转为二进制
for idx = 1 : block
    for pos_idx = 1 : num
        if info_idx > length(info)
            break; % 所有信息都已隐藏，则提前返回
        end
        linear_idx = (idx - 1) * 64 + pos(pos_idx, idx); % 线性索引计算
        Co_bin(linear_idx, end) = dec2bin(info(info_idx)); % 在 LSB 存储隐藏信息
        info_idx = info_idx + 1;
    end
    if info_idx > length(info)
        break; % 所有信息都已隐藏，则提前返回
    end
end

Co = zeros(size(Co));
for idx = 1 : numel(Co)
    if Co_bin(idx, 1) == '1' % 负数
        Co(idx) = bin2dec(Co_bin(idx, :)) - 256; % 补码处理
    else % 正数
        Co(idx) = bin2dec(Co_bin(idx, :));
    end
end

DC = Co(1, :); % 在系数矩阵中提取 DC 系数
DC_ans1 = DCencode(DC, DCTAB); % 对 DC 系数编码

AC = Co(2 : end, :); % 在系数矩阵中提取 AC 系数
```

```
AC_ans1 = [];  
for idx = 1 : block  
    AC_idx = AC(:, idx)'; % 每个块的 AC 系数（行向量）  
    AC_block = ACencode(AC_idx, ACTAB); % 对 AC 系数编码  
    AC_ans1 = [AC_ans1 AC_block]; % 拼合所有块的 AC 系数  
end  
  
comp = h * w * 8 / (length(DC_ans1) + length(AC_ans1)); % 计算压缩比  
fprintf("压缩比为%.3f\n", comp);  
  
image = zeros(h, w);  
  
DC_ans2 = DCdecode(DC_ans1, DCTAB, block); % 对 DC 系数解码  
AC_ans2 = ACdecode(AC_ans1, ACTAB, block); % 对 AC 系数解码  
im_ans2 = [DC_ans2; AC_ans2];  
  
% 提取隐藏的信息  
info1 = [];  
info_idx = 1;  
  
for idx = 1 : block  
    for pos_idx = 1 : num  
        if info_idx > length(info)  
            break; % 如果所有信息都已找到，则提前返回  
        end  
        info2 = dec2bin(im_ans2(pos(pos_idx, idx), idx)); % 转换为二进制  
        % 方便提取  
        info2 = bin2dec(info2(end)); % 提取隐藏在 LSB 的信息  
        info1 = [info1; info2];  
        info_idx = info_idx + 1; % 移动到下一个存储位置  
    end  
    if info_idx > length(info)  
        break; % 如果所有信息都已找到，则提前返回  
    end  
end  
  
acc = length(find(info == info1(1:length(info)))) / length(info); % 计算  
% 信息提取准确率  
fprintf("信息恢复准确率为%.2f%%\n", acc * 100);  
  
wb = w / 8;  
hb = h / 8;  
  
for i = 1 : hb
```

```
    for j = 1 : wb
        idx = (i - 1) * wb + j;
        Co_block = im_ans2(:, idx)'; % 获取当前块的系数向量
        im_block = zigzag_i(Co_block, 8, 8); % 使用反向 zigzag 扫描将向量转换为 8x8 矩阵

        im_block = im_block .* QTAB; % 逆向量量化
        im_block = idct2(im_block); % 逆 DCT 变换
        image((8 * i - 7) : 8 * i, (8 * j - 7) : 8 * j) = im_block; % 将块放置在完整图像中的正确位置
    end
end
image = uint8(image + 128); % 逆向预处理操作

% 计算 PSNR
MSE = sum(sum((double(image) - double(hall_gray)).^2)) / (w * h);
PSNR = 10 * log10(255^2 / MSE);

fprintf("PSNR 为%.3f\n", PSNR);

figure; imshow(hall_gray, 'InitialMagnification', 'fit'); title('原图');
figure; imshow(image, 'InitialMagnification', 'fit'); title('隐藏信息图');
```

### 方法三

```
load ('hall.mat');
load ('JpegCoeff.mat');

[h, w] = size(hall_gray);
im = double(hall_gray) - 128; % 图像预处理

Co = trans(im, QTAB); % 分块, DCT 变换, 量化

% 生成信息序列, 长度为块数 (w*h/64), 每个块隐藏 1 比特信息
info = double(randi([0, 1], 1, w * h / 64));
info = info * 2 - 1; % 转换为±1 序列: 0→-1, 1→1

% 隐藏信息到 DCT 系数中
for i = 1 : w * h / 64
    idx = find(Co(:, i)); % 找到当前块中非零系数的位置
    if isempty(idx)
        Co(2, i) = info(1, i); % 如果全为零, 在第一个 AC 系数位置隐藏信息
    else
```



```
        if idx(end) == 64
            Co(64, i) = info(1, i); % 如果最后一个系数非零，用信息位替换该系
数
        else
            Co(idx(end) + 1, i) = info(1, i); % 在最后一个非零系数后追加信
息
        end
    end
end

DC = Co(1, :); % 在系数矩阵中提取 DC 系数
DC_ans1 = DCEncode(DC, DCTAB); % 对 DC 系数编码

AC = Co(2 : end, :); % 在系数矩阵中提取 AC 系数
AC_ans1 = [];
for idx = 1 : w * h / 64
    AC_idx = AC(:, idx)';
    AC_block = ACencode(AC_idx, ACTAB); % 对 AC 系数编码
    AC_ans1 = [AC_ans1 AC_block]; % 拼合所有块的 AC 系数
end

comp = h * w * 8 / (length(DC_ans1) + length(AC_ans1)); % 计算压缩比
fprintf("压缩比为%.3f\n", comp);

image = zeros(h, w);
block = h * w / 64;

DC_ans2 = DCdecode(DC_ans1, DCTAB, block); % 对 DC 系数解码
AC_ans2 = ACdecode(AC_ans1, ACTAB, block); % 对 AC 系数解码
im_ans2 = cat(1, DC_ans2, AC_ans2);

% 从 DCT 系数中提取隐藏信息
info1 = zeros(1, w * h / 64);
for i = 1 : w * h / 64
    idx = find(im_ans2(:, i)); % 找到当前块中非零系数的位置
    if ~isempty(idx)
        info1(1, i) = im_ans2(idx(end), i);
    end
end

% 计算信息提取准确率
accuracy = length(find(info == info1)) / (w * h / 64);
fprintf("信息恢复准确率为%.2f%%\n", accuracy * 100);
```

```
wb = w / 8;
hb = h / 8;

for i = 1 : hb
    for j = 1 : wb
        Co_block = im_ans2(:, (i - 1) * wb + j)'; % 获取当前块的系数向量
        im_block = zigzag_i(Co_block, 8, 8); % 使用反向 zigzag 扫描将向量转换为 8x8 矩阵

        im_block = im_block .* QTAB; % 逆向量量化
        im_block = idct2(im_block); % 逆 DCT 变换
        image((8 * i - 7) : 8 * i, (8 * j - 7) : 8 * j) = im_block; % 将块放置在完整图像中的正确位置
    end
end
image = uint8(image + 128); % 逆向预处理操作

% 计算 PSNR
MSE = sum(sum((double(image) - double(hall_gray)).^2)) / (w * h);
PSNR = 10 * log10(255^2 / MSE);

fprintf("PSNR 为%.3f\n", PSNR);

figure; imshow(hall_gray, 'InitialMagnification', 'fit'); title('原图');
figure; imshow(image, 'InitialMagnification', 'fit'); title('隐藏信息图');
```

### (3) 运行结果

三种方法信息恢复准确率均为 100%，说明在变换域上隐藏信息的方法抗 JPEG 编码能力强。

从压缩比上看，不加任何处理的大礼堂图片经过 JPEG 编码，压缩比是 6.425，三种变换域信息隐藏方法均使压缩比减小，说明隐藏信息需要更多存储空间，其中需要的额外空间大小方法三<方法一<方法二。

从图像质量上看，不加任何处理的大礼堂照片经过 JPEG 解码，PSNR 是 31.187dB，三种变换域信息隐藏方法均使 PSNR 减小，说明隐藏信息后图像还原的质量变差，失真变大，其中图像还原的性能方法三>方法二>方法一。

对于嵌密方法的隐蔽性，根据直观观察，方法一和方法二的含密图像有明显颗粒感，隐蔽性较差；方法三的含密图像整体无明显违和感，隐蔽性较好。

## 方法一

```
>> HW2_3_2_1
```

压缩比为5.778

信息恢复准确率为100.00%

PSNR为24.718

原图



隐藏信息图



## 方法二

```
>> HW2_3_2_2
```

压缩比为5.698

信息恢复准确率为100.00%

PSNR为26.551

原图



隐藏信息图



## 方法三

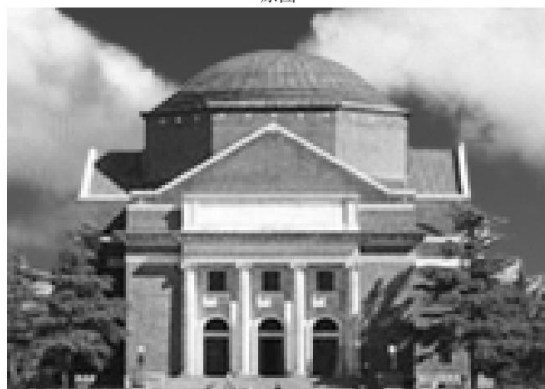
```
>> HW2_3_2_3
```

压缩比为6.192

信息恢复准确率为100.00%

PSNR为28.988

原图



隐藏信息图



## 五、 实验总结

本次 MATLAB 大作业通过对处理图像的基本知识的介绍和引入，一步步指导我结合信号与系统的知识撰写 MATLAB 代码，实现了图像压缩编码、解码、信息隐藏的全过程。我对图像的三个通道，编码部分分块、DCT 变换、量化、熵编码等操作，在空域和变换域上隐藏信息的常见方法有了更深入的理解，也了解了评价图像的两个基本指标：压缩比和 PSNR。通过本次实验，我也对图像处理形成了基本认知，并提升了我撰写 MATLAB 代码的能力，收获颇丰。